

Universidad Mariano Gálvez de Guatemala
Facultad de Ingeniería de Sistemas de Información
Inteligencia Artificial



Proyecto RAG

Tomás Pedro Pascual Mateo

Carne: 2590-21-10073

Plan Diario Vespertino Virtual

Sección "1"

24 de abril, 2026

Reporte Tecnico

Sistema RAG - Asistente de Códigos de Error BMW

Proyecto Individual | Inteligencia Artificial

1. Descripción del Proyecto

Sistema RAG (Retrieval-Augmented Generation) para consulta de códigos de error OBD-II en vehículos BMW. El sistema recupera información relevante de una base de conocimiento local y genera respuestas con evidencia y fuentes citadas.

2. Dataset

2.1 Estructura

- 15 documentos TXT organizados por sistema del vehiculo
- 90 codigos de error reales (OBD-II y especificos BMW)
- Cada codigo incluye: significado, causa probable, sintomas y solucion

2.2 Sistemas cubiertos

Sistema	Documentos	Codigos
Motor	3	18
Transmision	2	12
Frenos/ABS	2	12

Electrico/Bateria	2	12
Airbag/Carroceria	2	12
Emisiones	2	12
Direccion	2	12
TOTAL	15	90

3. Pipeline RAG

3.1 Stack tecnologico

- Embeddings: sentence-transformers / all-MiniLM-L6-v2 (local)
- Base vectorial: FAISS (local)
- LLM: Groq API / llama-3.3-70b-versatile
- Interfaz: Streamlit
- Lenguaje: Python 3.11.9

3.2 Flujo del sistema

- 1. Usuario ingresa una pregunta o codigo de error
- 2. Se genera un embedding de la consulta con all-MiniLM-L6-v2
- 3. FAISS busca los top-k fragmentos mas similares
- 4. Se construye un prompt con el contexto recuperado
- 5. Groq (llama-3.3-70b) genera la respuesta citando fuentes

- 6. Se retorna respuesta + fragmentos utilizados + fuente

4. Metricas de Evaluacion

4.1 Golden Set

12 preguntas de prueba con respuesta esperada y fuente esperada definidas manualmente (golden_set.json).

4.2 Metricas implementadas

- Precision@k: proporcion de fragmentos recuperados que provienen de la fuente esperada
- Recall@k: indica si al menos un fragmento de la fuente esperada fue recuperado en los primeros k resultados

4.3 Resultados base (chunk=500, k=5)

Metrica	Valor	Interpretacion
Precision@5	0.2333	23% de fragmentos relevantes
Recall@5	0.5833	58% de preguntas con fuente correcta
Latencia promedio	2.95s	Por consulta

5. Experimentos

5.1 Experimento 1 - Tamano de Chunk (k=5 fijo)

Chunk Size	Fragmentos	Precision@5	Recall@5
300	353	0.2333	0.5833
500	199	0.2333	0.5833
600	164	0.2000	0.5000

Decision: chunk_size=300 ofrece mejor recall con menor latencia (1.22s vs 3.09s). Se selecciona como configuracion final.

5.2 Experimento 2 - Top-k (chunk=500 fijo)

Top-k	Precision@k	Recall@k	Latencia
k=3	0.2500	0.4167	1.79s
k=5	0.2333	0.5833	2.95s
k=7	0.2738	0.8333	3.25s

Decision: k=5 ofrece el mejor balance entre recall (0.58) y latencia (2.95s). k=7 mejora recall pero aumenta latencia un 10%.

6. Limitaciones

- Algunos codigos con nombres similares generan confusion en el retrieval
- El modelo de embeddings es en ingles, los documentos en espanol pueden afectar precision
- El sistema depende de disponibilidad de la API de Groq
- No implementa memoria de conversacion entre consultas

7. Conclusiones

- El sistema RAG responde correctamente preguntas sobre codigos BMW con Recall@5 = 0.58
- La configuracion optima es chunk_size=300 con k=5
- FAISS con sentence-transformers funciona eficientemente para este dominio tecnico
- El sistema muestra siempre las fuentes utilizadas, cumpliendo el requisito de grounding